

M02 • TECHNICAL DESIGN DOCUMENT

Event and Operation Foundation

Durable identity, immutable evidence, deterministic projection, and recovery contracts

Product: HaloFlow Medical Solution
Status: Approved design baseline — implementation and production-readiness gates remain
Version / date: 0.3 • 30 August 2026
Requirements basis: M02 Detailed Requirements — Event and Operation Foundation, Approved 1.0
Primary dependency: M01 Technical Design — Tenant Context and Data Access, Draft 0.2

Table of Contents

Executive summary..... 4

1. Goals, non-goals, and constraints..... 4

 1.1 Goals 4

 1.2 Non-goals 4

 1.3 Hard constraints..... 5

2. Architecture decisions and approval status..... 5

3. System context and component ownership 6

 3.1 Components 7

 3.2 Module handoffs..... 7

4. Domain model and physical data design..... 8

 4.1 Tenant-local tables 8

 4.2 operation_registry 8

 4.2.1 Business-key version control 9

 4.3 patient_events 9

 4.4 Immutable links and canonical acceptance 11

 4.5 Shared reference and control data 11

 4.6 Required indexes and constraints 12

5. Event contracts and producer validation..... 13

 5.1 Event level model..... 13

 5.2 Contract descriptor 13

5.3 Validation order	14
6. Public service interfaces.....	14
6.1 Command/result contracts	15
6.2 Transaction contract.....	15
7. External side-effect workflow	16
7.1 beginOperation algorithm	16
7.2 Adapter boundary.....	17
7.3 Acceptance binding and conflicts	17
7.4 Indeterminate and resend	17
8. Inbound observation and callback workflow	17
8.1 Observation transaction	17
8.2 Verified delivery/business outcomes	18
8.3 Terminal conflict.....	18
9. Deterministic projection design	19
9.1 Pure projection algorithm	19
9.2 Explanation codes.....	20
9.3 Hybrid cache and rebuild	20
10. Crash-durable control handoffs.....	21
10.1 Outbox record.....	21
10.2 Delivery semantics	22
11. Query, pagination, and authorization	23
11.1 Operation projection reads.....	23
11.2 Bulk projection.....	23
11.3 Subject timeline	23
11.4 Stale-intent evidence query	23
12. Corrections, manual resolution, and privileged evidence	24
13. Security and privacy design.....	25
13.1 Roles and grants	25
13.2 Threat controls.....	27
13.3 Business-key HMAC custody and lifecycle.....	28
13.4 Minimum-necessary metadata	28

13.5 Retention-locked export evidence	29
14. Failure behavior and error taxonomy	29
15. Observability and service objectives	30
16. Reliability, performance, and capacity.....	30
17. Verification strategy and release gates	31
17.1 Mandatory negative scenarios.....	32
18. Migration, rollout, and rollback.....	33
18.1 Rollback.....	34
19. Open decisions and prerequisites.....	34
20. Requirement traceability	34
Appendix A. Repository and implementation controls.....	36
Appendix B. Source basis and assumptions	37
Appendix C. Design review checklist.....	38
Appendix D. v0.3 review disposition and approval	38

Downstream contracts: M03, M05, M06, M09, M11, M12; event consumers across M07–M10

Design owner: Platform Engineering (assignment to be confirmed)

Approval status. Approved as the M02 implementation design baseline. This revision closes review findings T1-T8 and V1-V5 without granting raw runtime table mutation. Schema freeze still requires ADR-011 and the named content gates; pilot and production remain subject to provider, retention, service-objective, recovery, and operational-readiness evidence.

Document purpose

Translate the approved M02 requirements into implementable components, schemas, APIs, transaction boundaries, projection rules, handoff mechanics, security controls, failure behavior, observability, rollout steps, and verification evidence. The approved M02 requirement controls if this design conflicts with it.

Executive summary

M02 provides one tenant-aware foundation for every externally visible side effect and material inbound observation. An owning workflow first acquires a stable `operation_id` and commits an immutable requested event through the M01 tenant transaction gateway. Only then may it call a provider. Submission, delivery, business outcome, observation, conflict, correction, and manual-resolution facts are appended separately; historical evidence is never updated or deleted by runtime identities.

The design separates evidence from coordination. Tenant-local `patient_events` and related immutable records are authoritative. A versioned `ProjectionEngine` derives current state and maintains a rebuildable cache for bounded bulk reads. Tenant-local handoff outbox records make M05 work, M06 reconciliation, and M11 signals crash durable; dedicated dispatchers deliver them at least once using deterministic tenant-scoped keys.

Recommended baseline. Use a hybrid projection: compute with one pure, versioned algorithm; update the cache in the same tenant transaction as each append; rebuild from immutable events when the version changes or cache integrity is uncertain. The cache never becomes evidence and never permits external replay.

1. Goals, non-goals, and constraints

1.1 Goals

1. Guarantee one stable `operation_id` across retries, restart, deployment, reconciliation, and worker takeover.
2. Prevent an external side effect unless durable intent has committed in the authorized tenant transaction.
3. Retain every distinct verified fact while collapsing equivalent duplicate processing.
4. Derive one explainable current state deterministically from immutable events, including corrections and conflicts.
5. Provide crash-durable, idempotent M05/M06/M11 handoffs without cross-schema foreign-key fiction.
6. Keep event metadata, central telemetry, replay, and privileged actions tenant-safe and minimum necessary.
7. Give M03–M12 stable contracts that can roll forward without reinterpreting historical evidence.

1.2 Non-goals

8. Implementing webhook verification, provider routing, queue state machines, batch scheduling, or clinic-facing UI.
9. Using an event broker or Cloud Logging as the authoritative clinical/audit event store.
10. Providing general cross-tenant event queries through runtime credentials.
11. Automatically resending an operation whose external outcome is indeterminate.

12. Storing raw provider payloads, message bodies, unrestricted free text, or clinical data in generic event metadata.
13. Treating append_sequence as commit order, a global cursor, or an incremental replay watermark.

1.3 Hard constraints

Constraint	Design consequence
M01 tenant boundary	Every append, projection, timeline, and correction uses TenantContext and TenantTransactionGateway; repositories never accept schema names.
Append-only authority	Runtime executes approved procedures and may read authorized views; it has no raw evidence-table mutation grants.
Intent before action	beginOperation commits the stable operation and requested event before an adapter may execute.
At least once	Producer/dispatcher execution may repeat; deterministic keys make the effective append, case, or work item converge.
Explainable projection	Every state includes controlling event IDs, explanation code, algorithm version, and freshness.
Fail safe	Unknown contracts, invalid status scope, cross-tenant links, ambiguous precedence, or untrusted terminal evidence never project success.

2. Architecture decisions and approval status

ID	Decision	Status	Rationale / consequence
D-01	Tenant-local patient_events remains the immutable source of truth	Required	Preserves M01 isolation and the approved physical table name.
D-02	Tenant-local operation_registry acquires stable identity and business dedupe	Proposed baseline	Uses one database-controlled active write-key version so rolling deployment cannot fork a logical operation.
D-03	Six event levels: intent, submission, delivery, business_outcome, observation, correction	Proposed for ADR-011	Keeps three-stage side effects while representing non-side-effect facts explicitly.
D-04	Hybrid projection: pure engine plus transactionally refreshed, rebuildable cache	Proposed baseline	Serializes per-operation appends before folding the complete event set; derived state remains non-authoritative.
D-05	Tenant-local event_handoff_outbox for M05/M06/M11	Proposed baseline	Avoids dual-write gaps and enforces at-least-once convergence.
D-06	Two explicit producer dedupe forms plus event_id retry	Proposed baseline	Collapses equivalent retries while retaining independently sourced conflicts.

ID	Decision	Status	Rationale / consequence
D-07	Canonical acceptance binding is separate from event evidence	Proposed baseline	PK(operation_id) always applies; cross-operation provider identity uniqueness is enforced only by a declared capability scope.
D-08	Reference codes and contract descriptors are shared control data; no PostgreSQL ENUM	Required	Allows additive rollout, retirement, validation, and compatibility checks.
D-09	No cryptographic hash chain in v1	Approved v1 baseline	Use append-only grants, database audit, PITR, retention-locked signed exports, and integrity reconciliation; revisit if policy requires cryptographic continuity.
D-10	Pilot replay restarts one declared bounded tenant/range	Required baseline	append_sequence orders a complete set but is not safe as a resume watermark.

Decision boundary. D-02 through D-07 are implementation decisions in this approved baseline and must be incorporated into ADR-011 or an equivalent approved architecture record before schema freeze. D-09 is approved for the v1 baseline with retention-locked signed exports and must be revalidated before pilot if policy changes.

3. System context and component ownership

M02 component architecture and authority boundaries

Immutable tenant evidence is authoritative; projections and control handoffs are derived.

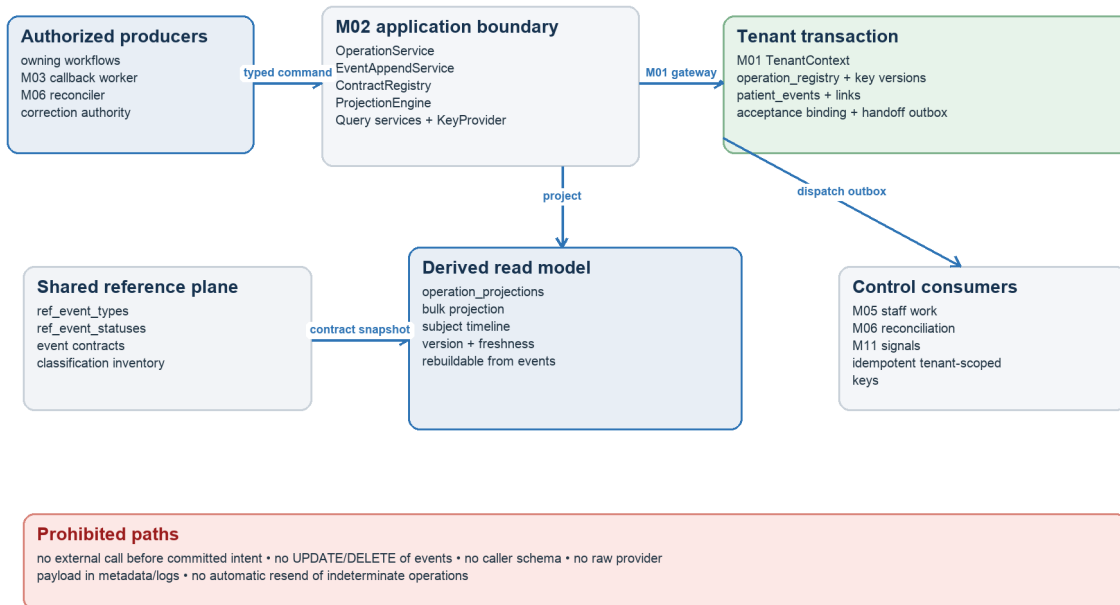


Figure 1. M02 component architecture and authority boundaries

3.1 Components

Component	Responsibility	Not allowed
OperationService	Acquire or recover operation_id; persist operation registry row and requested event atomically; create authorized resend linkage.	Provider I/O, schema selection, or mutable operation status.
EventAppendService	Validate command, contract, status scope, authority, links, metadata, timestamps, dedupe, and append transaction.	Raw provider payload storage or direct shared-control writes.
ContractRegistry	Load active type/status/contract descriptors; cache versioned snapshots; fail closed on incompatible or missing entries.	Interpreting unknown codes or rewriting historical contract meaning.
KeyProvider	Resolve approved tenant/version HMAC key material through the configured KeyProvider and fail closed when a required version is unavailable. Cloud KMS envelope integration and its cache lifecycle are OI-009 follow-up; LocalDevKeyProvider is non-production only.	Database mutation, cross-tenant key enumeration, key export, logging secrets, or choosing a replacement version.
ProjectionEngine	Pure deterministic fold over a complete event set after correction and duplicate handling; produce explanations.	External calls, queue mutation, use of received_at to invent terminal precedence.
ProjectionRepository	Maintain and query derived operation_projections with version/freshness; support rebuild and bulk paging.	Treating cache rows as source evidence.
TimelineQueryService	Return authorized subject facts with correction/provenance and masked metadata under stable pagination.	Cross-tenant or unbounded queries.
HandoffDispatcher	Lease tenant outbox rows and deliver tenant-scoped commands/signals at least once to M05/M06/M11.	Marking delivered before consumer acknowledgement or embedding PHI.
IntegrityVerifier	Reconcile operation, event, binding, projection, outbox, and privileged-audit invariants; alert on unexplained defects.	Treating normal sequence gaps as tampering.

3.2 Module handoffs

Module	M02 provides	Consumer owns
M03	recordObservation / recordOutcome contracts, provider-source dedupe, correlation trust validation.	HMAC verification, durable inbox, claim/lease/reclaimer, routing resolution.
M05	At-least-once failure/conflict/manual-review command with deterministic key and source evidence IDs.	One effective staff work item, mutable lifecycle, assignment, SLA, resolution UX.
M06	Stable operation, indeterminate evidence, bulk projection, stale-intent evidence query, and tenant-bound reconciliation command.	Scheduling, adapter lookup, leases, reconciliation-case lifecycle, retry policy.
M10	Bulk projection and subject timeline with version, freshness, explanation, and masking.	Clinic-facing presentation, authorization UX, pagination experience.
M11	PHI-safe append/projection/handoff signals and integrity metrics.	Correlation governance, log pipeline, dashboards, alerts, on-call runbooks.
M07–M09/M12	Versioned producer contracts for domain-specific event types and outcomes.	Business rules, subject validity, failure reason vocabulary, extension installation.

4. Domain model and physical data design

4.1 Tenant-local tables

Table	Purpose	Mutability / role
operation_registry	Stable operation identity, action/owner, dedupe fingerprint, subject anchor, resend linkage, creation provenance.	Insert-only through approved procedures; one row per logical operation.
operation_key_versions	Active, target, and retained business-key/HMAC versions per owner and action; no secret material.	V1 exposes no mutation gateway for key-control state. The lock owner holds the owner-accepted UPDATE(updated_at) privilege for locking SELECT, while its gateways issue no UPDATE. Rotation state mutation and its controlled procedure are OI-009 follow-up.
patient_events	Authoritative immutable facts, nullable patient_id, and total-order append_sequence within the tenant schema.	Insert/read through approved procedures; no updated_at.
patient_event_links	Typed immutable links from an event to patient, appointment, log, batch/run, message, document, or approved subject/source.	Inserted in the same transaction as the event.
operation_acceptance_bindings	One canonical provider/account/external identity per operation; first verified acceptance wins until authorized correction.	Insert once; correction creates replacement evidence through privileged workflow.
operation_projections	Derived current state, controlling IDs, explanation, engine version, source high-water snapshot, and refreshed_at.	Mutable/rebuildable; not evidence.
event_handoff_outbox	Crash-durable commands/signals to M05, M06, and M11 with deterministic dedupe key and delivery lease state.	Mutable delivery state; payload structural and PHI-minimum.

4.2 operation_registry

Column	Type / nullability	Rule
operation_id	uuid • PK	Cryptographically strong UUID generated before insert; provider idempotency key where supported.
owner_service	varchar(64) • required	Controlled producer identity; immutable.
action_code	varchar(96) • required	Active event-contract action family.
business_key_fingerprint	bytea • required	HMAC-SHA-256 of canonical owning-module business key; key version stored separately; no raw PHI.
business_key_version	smallint • required	Database-approved active write version used for this row; rotation never changes existing operation_id.
subject_type / subject_id	varchar(48) / uuid • required	Authorized tenant-local anchor; validated before insert.

Column	Type / nullability	Rule
resend_of_operation_id	uuid • nullable FK	Only manual authorized resend; cannot self-reference; chain cycle rejected.
correlation_id	uuid • required	Request/job scope at creation; not operation identity.
producer_version	varchar(64) • required	Deployment provenance.
created_at	timestampz • DB default	Immutable insertion time.

Uniqueness and version control. UNIQUE(owner_service, action_code, business_key_version, business_key_fingerprint) protects each version. In v1, beginOperation calls m02_begin_key_scope to lock the exact operation_key_versions row FOR KEY SHARE and obtain active/retained identifiers and generation. It computes candidates for every retained version before active-version insertion and fails closed when any required key is unavailable. A resend uses a distinct owning-module business key and resend_of_operation_id. Rotation orchestration, m02_rotate_key_version execution and live producer compatibility gating are deferred under D-11.10 to OI-009; their future locking protocol must preserve cross-version lookup continuity and prevent cross-version forks.

4.2.1 Business-key version control

Field	Rule
owner_service / action_code	Composite primary key; one control row for each owning business-key contract in the tenant schema.
active_write_version	Exactly one version is accepted for new registry rows. In v1 m02_begin_key_scope locks the exact control row FOR KEY SHARE and validates active/retained identifiers and generation; service-side KeyProvider availability checks fail closed. Live producer registration checks are deferred to OI-009.
retained_lookup_versions	Ordered bounded versions used only for lookup; each must have a recoverable tenant key envelope.
target_write_version / compatibility_generation	Retained schema/provenance for the future prepared version and compatibility generation; live producer compatibility and activation machinery are deferred to OI-009 and are not a v1 gateway assertion.
generation	Monotonic active control generation returned and validated in v1. Binding to the reference snapshot remains part of the approved service contract; producer startup declarations and rotation audit are OI-009 follow-up.
updated_at / updated_by	Database timestamp and privileged workload identity. V1 retains the owner-accepted lock-owner UPDATE(updated_at) residual, characterized separately from the registry UPDATE rejection; gateway bodies never mutate it. Rotation is unavailable in v1 and its audit/mutation protocol is OI-009 follow-up.

4.3 patient_events

Column	Type / nullability	Rule
event_id	uuid • PK	Generated before insert and reused for retry/commit-ambiguity lookup.

Column	Type / nullability	Rule
append_sequence	bigint identity • unique	Tenant-local total-order tiebreaker over a complete visible set; not commit order/cursor.
operation_id	uuid • nullable FK	Required for external side effects, failure/conflict, correction of operation facts, and manual resolution.
patient_id	uuid • nullable FK	Approved primary patient anchor when known; at least patient_id or one approved subject/source link is required.
event_type / contract_version	varchar(128) / integer	Active shared type and exact immutable contract version.
event_level / status	varchar(32) / varchar(64)	Validated combination; status scope must be stored_event or both.
channel / direction	varchar(32) • nullable	Required when declared by contract.
correlation_id	uuid • required	Trusted entry-point request/job value; validated but never used as durable identity.
causation_event_id	uuid • nullable FK	Prior tenant event that caused this assertion.
provider / provider_account	varchar(64) • nullable	Required whenever external/provider identifiers appear.
external_id / provider_event_id	varchar(256) • nullable	Namespaced; normalized and bounded; intent/rejected/indeterminate normally omit external_id.
source_namespace / source_event_id	varchar(64) / varchar(256) • nullable	Verified producer-source dedupe identity.
producer_dedupe_key	varchar(256) • nullable	Contract-defined retry identity when no verified source event ID exists.
content_fingerprint	bytea • required	SHA-256 over canonical immutable fields, links, and metadata; equivalent-key retries must match.
producer_service / producer_version	varchar(64) / varchar(64)	Authorized writer and deployment provenance.
actor_id / workload_id / auth_method	uuid / uuid / varchar(32)	Exactly one principal kind plus authentication evidence; correction may require actor.
occurred_at / source_time_quality	timestamptz / varchar(24)	Source time plus trusted, estimated, absent, or invalid quality.
received_at / created_at	timestamptz / timestamptz	Platform receipt and DB insertion; created_at DB assigned.
corrected_event_id	uuid • nullable FK	Required only by correction/resolution contract; same tenant.
correction_reason / correction_scope	varchar(64) / varchar(32)	Controlled reason and metadata_only or asserted_fact semantics.
failure_reason_code / resolution_path_code	varchar(64) • nullable	Controlled owning-module vocabulary; no unrestricted explanation.
metadata	jsonb • required default {}	Contract JSON Schema; 8 KiB encoded maximum baseline; allowlisted and PHI-minimum.

4.4 Immutable links and canonical acceptance

Record	Key fields	Integrity
patient_event_links	event_id, link_type, link_id, relationship	PK(event_id, link_type, link_id, relationship); same-tenant resolution through an allowlisted resolver. A deferred database constraint requires patient_id or at least one approved link at COMMIT.
operation_acceptance_bindings	operation_id, provider, provider_account, identity_namespace, external_id, uniqueness_scope, source_event_id	PK(operation_id) always. A partial unique index on provider/account/namespace/external_id applies only when the immutable provider capability declares account_namespace uniqueness. Otherwise no cross-operation uniqueness is inferred. Competing identity creates acceptance_conflict evidence and an M05 outbox row.

4.5 Shared reference and control data

Shared object	Design contract	Writer
ref_event_types	Stable code, action family, display label, active flag, current contract version, permitted producer/level metadata.	Migrations/provisioning only.
ref_event_statuses	Stable code, status_scope, terminality, semantic level, precedence rank, compatible-outcome class, active flag.	Migrations/provisioning only.
event_contracts	Versioned JSON Schema and declarative constraints: required links, statuses, producers, metadata, projection behavior.	Migrations/provisioning only; old versions retained.
provider_capabilities	Versioned external-ID namespace and uniqueness scope, lookup/idempotency/time capability evidence, and activation state.	Controlled integration migration; old versions retained.
producer_key_capabilities	OI-009 follow-up requirement; not a v1 delivery or activation precondition under D-11.10. PHI-free live workload registration: producer identity, deployment generation, supported key versions, heartbeat expiry, and attestation source.	OI-009 follow-up requirement; not a v1 delivery or activation precondition under D-11.10. Producer startup/heartbeat identity; rotation reads through the definer gateway.
reconciliation_cases	tenant_id + operation_id unique; mutable M06 scheduling/lease state; verified tenant event/work links.	M06 controlled identity.
access_audit_log	Privileged correction, export, disposition, integrity override, and break-glass evidence.	M01/M11 approved identities.

4.6 Required indexes and constraints

1. patient_events: PK(event_id); UNIQUE(append_sequence); indexes (operation_id, append_sequence), (patient_id, created_at, event_id), (correlation_id, append_sequence), and link-driven timeline indexes. The named partial index idx_patient_events_stale_intent_scan on (created_at, operation_id, event_id) WHERE event_level='intent' AND operation_id IS NOT NULL serves the bounded stale-intent scan; the operation index serves its submission anti-join. JSON scans are not the normal query path; patient, link, and stale-intent plans are load-tested.
2. Equivalent source events: partial UNIQUE(producer_service, source_namespace, source_event_id) where source_event_id is not null.
3. Producer-generated facts without a source event: partial UNIQUE(producer_service, producer_dedupe_key) where producer_dedupe_key is not null; the key is a structured fingerprint defined by the contract.
4. A duplicate key is success only if the immutable content fingerprint matches. A mismatch raises DEDUPE_CONFLICT, retains attempted structural evidence outside patient data, and requires investigation.
5. A DEFERRABLE INITIALLY DEFERRED constraint trigger named trg_patient_event_subject_required rejects COMMIT when patient_id is null and no approved patient_event_links row exists. Foreign keys and validation triggers also reject cross-operation correction, causation, resend, and acceptance links and statuses whose scope is projection_only.
6. The acceptance binding partial unique index applies only to rows whose versioned provider capability sets uniqueness_scope=account_namespace; the stored namespace and scope are immutable and validated before insert.
7. Runtime has EXECUTE on purpose-specific M02 gateways and SELECT-only reviewed read views/functions. In each creation transaction revoke inherited runtime INSERT, UPDATE and DELETE on operation_registry, operation_key_versions, patient_events, patient_event_links, operation_acceptance_bindings, operation_projections and event_handoff_outbox; assert effective TRUNCATE absent. Remove runtime table SELECT only with the complete approved replacement read path. Remove inherited runtime USAGE and SELECT on actual M02-created sequences at creation; grant sequence USAGE only to the execution owner that allocates values. Application code orchestrates one TenantTransactionGateway transaction; gateways enforce local invariants and narrow mutations.

8. Because PostgreSQL locking SELECTs require UPDATE privilege, haloflow_runtime never issues them directly. m02_lock_operation(operation_id) and m02_begin_key_scope(owner_service, action_code) are SECURITY DEFINER gateways owned by the dedicated NOLOGIN haloflow_m02_lock_owner role. Each gateway uses invariant tenant-object-unqualified body references and pins search_path to pg_catalog, the validated tenant schema and pg_temp last; M01 validates tenant routing, principal, expiry and per-statement capability before invocation, while the SQL gateway checks the exact local row and mutation invariants without authenticating tenant context, treats any app.tenant_id check only as well-formedness, returns minimum structural data, and has PUBLIC EXECUTE revoked. Locks acquired inside either gateway remain held by the caller's TenantTransactionGateway transaction until COMMIT or ROLLBACK.

5. Event contracts and producer validation

5.1 Event level model

Level	Use	Examples / rules
intent	Committed decision to attempt an external side effect.	{action}_requested; requested/pending; operation_id required; external_id forbidden.
submission	Immediate normalized call result.	{action}_accepted, _rejected, _submission_indeterminate; acceptance identity only when verified.
delivery	Verified provider delivery outcome.	{action}_delivered, _delivery_failed, _delivery_conflict; callback/lookup provenance required.
business_outcome	Verified domain result after submission.	EMR/domain completion or failure; typed business link and evidence source required.
observation	Accepted inbound/material fact not representing an outbound attempt.	patient_reply_received, fax_received, sms_stop_received; completed status; owning state change is atomic.
correction	Authorized correction, supersession, or manual resolution.	corrected_event_id, actor/workload, reason, scope, evidence source; original remains queryable.

5.2 Contract descriptor

```

EventContract {
  event_type, version, action_family, event_level,
  allowed_statuses[], terminality, compatible_outcome_class?, precedence_rank,
  required_links[], optional_links[], metadata_json_schema, metadata_max_bytes,
  authorized_producers[], required_provenance[], correction_policy,
  projection_rule_id, effective_from, retired_at?
}

```

ContractRegistry loads one immutable snapshot per tenant transaction. Startup fails if core seed codes or descriptors are absent, inactive without an approved rollout state, incorrectly scoped, or unsupported by the deployment. Owning modules install extension contracts before enabling producers. Additive schema changes receive a new contract version only when validation or projection

behavior changes; historical rows retain their stored version. The issued M02_CP0_Contract_Baseline_v1.0.md publication index pins the approved physical/test contract artifacts and their explicit supersession order, including exact closed envelopes, synthetic rules, read manifests, SignerProvider interface and provisional limits. Historical proposal-status labels in the immutable review attachments do not override that issuance status or its stated precedence. Production clinical/support fields, evidence resolvers and policies require separately registered exact owning-module contracts.

5.3 Validation order

1. M01 and the service validate TenantContext, capability, producer identity, context expiry, and operation/subject ownership before the protected database work. SQL gateways separately enforce local row, link and mutation invariants; they do not independently authenticate tenant context. M02 v1 requires principal.id to be canonical UUID text. Actor/workload kind selects exactly one event identity column; the correction/resolution gateway derives matching audit principal_id and principal_kind from that same canonical value and rejects inconsistent attribution. The public command cannot override service-derived principal, execution or request attribution. Purpose is the intersection of the configured M01 allowlist and treatment/payment/operations; M01 owns the purpose domain and M02 its audit action/resource/outcome mappings. Reject oversized request_id/auth_method before conversion, never truncate. SQL equality preserves attribution consistency, not independent authentication; existing privileged-service and runtime audit DML limitations remain.
2. Resolve the exact active event type and contract version; verify producer, event level, status scope, and legal combination.
3. Validate operation, causation, correction, provider namespace, subject/source links, and same-tenant references.
4. Validate occurred_at quality, metadata JSON Schema and size, controlled failure/correction codes, and prohibited fields.
5. Compute immutable content fingerprint and dedupe identity; return the prior row only when fingerprints match.
6. Append event, related links, binding/projection changes, and required outbox records in one tenant transaction.

6. Public service interfaces

```
class EventFoundation:
    begin_operation(ctx, action, subject, business_key, links) -> OperationStart
    record_submission_outcome(ctx, operation_id, outcome) -> AppendResult
    record_outcome(ctx, operation_id, outcome) -> AppendResult
    record_observation(ctx, observation, apply_state_change) -> AppendResult
    append_correction(ctx, corrected_event_id, correction) -> AppendResult
    record_manual_resolution(ctx, operation_id, resolution) -> ResolutionResult
    get_operation_projection(ctx, operation_id, consistency) -> ProjectionResult
    list_operation_projections(ctx, selector, cursor, page_size) -> ProjectionPage
    list_stale_intents(ctx, cutoff, selector, cursor, page_size) -> StaleIntentPage
    list_subject_timeline(ctx, subject, cursor, page_size, view) -> TimelinePage
```

6.1 Command/result contracts

Contract	Required fields	Behavior
OperationStart	operation_id, requested_event_id, created_or_existing, projection_version	Returned only after commit; an existing operation is returned when the business fingerprint matches.
SubmissionOutcome	outcome kind, provider/account, external ID when known, source provenance, occurred_at quality, producer dedupe	Adapter-normalized; cannot encode delivery or unrestricted error text.
Observation	event type/status, subject/source links, trusted correlation, source dedupe, accepted state-change callback	Event and owning accepted state mutation share one tenant transaction.
AppendResult	event_id, appended_or_existing, operation_id?, projection summary, handoff state	Equivalent duplicate is success; commit ambiguity is resolved by lookup before retry.
ProjectionResult	status, terminal, conflict/indeterminate flags, controlling IDs, explanation, version, freshness	No metadata body by default; unsupported/incomplete is explicit.
ProjectionPage	items, stable cursor, page version, generated_at, per-item error	One tenant, bounded selector, one projection-engine version per page.
StaleIntentPage	requested event, operation/action, age, provider lookup hints, signed cursor	Authoritative anti-join over intents with no qualifying submission fact; discovery does not change projection.
TimelinePage	ordered immutable facts, correction relationships, masked provenance, cursor	No hidden metadata by default; stable order uses chronology plus append_sequence.

6.2 Transaction contract

Every write interface is invoked through TenantTransactionGateway. The service receives a scoped repository handle and contract snapshot, never a pool connection or schema identifier. The caller may supply an owning state-change callback only where the interface explicitly supports atomic domain mutation; nested tenant gateway calls and tenant changes inside the callback are rejected.

Commit meaning. Success means the authoritative Cloud SQL transaction committed. M11 signal delivery may lag or fail without reversing that truth. When commit outcome is uncertain, the caller receives APPEND_COMMIT_UNKNOWN and must resolve by event_id or dedupe key before attempting the same logical append.

7. External side-effect workflow

External side effect: intent before action and durable recovery

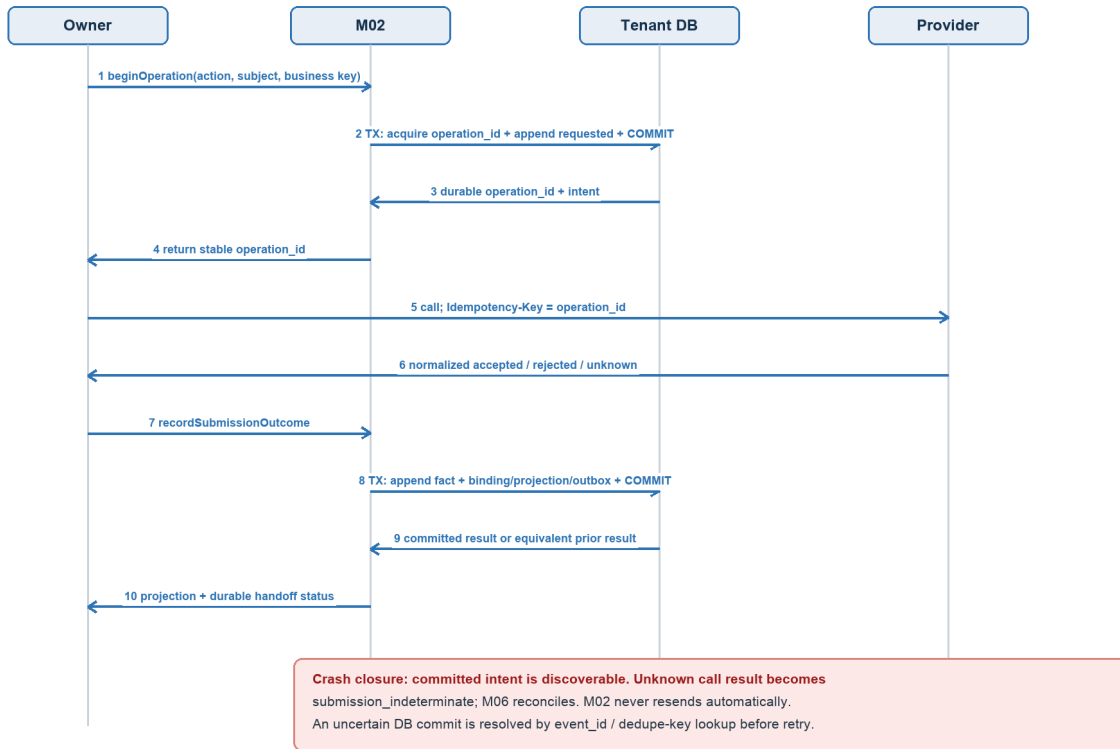


Figure 2. External side-effect intent, call, outcome, and crash closure

7.1 beginOperation algorithm

1. Canonicalize the owning-module business key and call `m02_begin_key_scope(owner_service, action_code)` inside the current `TenantTransactionGateway` transaction. After M01 validates tenant routing, principal, expiry and per-statement capability, the SECURITY DEFINER gateway checks local invariants, locks the exact `operation_key_versions` row FOR KEY SHARE and returns the database-controlled active/retained version identifiers and generation needed for tenant-isolated HMAC candidates without authenticating context or verifying live producer registration in v1, whose heartbeat machinery remains deferred under D-11.10/OI-009; the raw business key is not persisted.
2. Look up every retained candidate before insert, then INSERT `operation_registry` under the single active write version with ON CONFLICT against that version's fingerprint. On conflict, compare immutable action, owner, subject, and resend semantics; mismatch is a security-significant DEDUPE_CONFLICT.
3. Append `{action}_requested` with the acquired `operation_id` and typed links in the same transaction. The requested event uses a deterministic producer dedupe key derived from `operation_id` plus contract type.

- Refresh the derived projection to requested/pending and enqueue the non-authoritative M11 signal in the same transaction.
- Commit. Only a committed OperationStart authorizes the adapter call.

7.2 Adapter boundary

Raw adapter result	Normalized M02 result	Event
Provider accepted with verified identity	accepted(provider, account, external_id, source)	{action}_accepted
Definitive validation/business rejection	rejected(failure_reason_code, source)	{action}_rejected
Timeout, connection loss, malformed/ambiguous response	submission_indeterminate(reason_code, source)	{action}_submission_indeterminate
Unknown provider status	unsupported_mapping(raw_code_hash, adapter_version)	No success event; compatibility/reconciliation path

Notifyre. For Notifyre, the UUID operation_id is transmitted byte-for-byte as the provider idempotency key on every attempt, restart, deployment, and takeover. No HMAC, prefix, suffix, or derivation is permitted.

7.3 Acceptance binding and conflicts

On an accepted result, EventAppendService attempts to insert operation_acceptance_bindings in the same transaction as the accepted event. The first verified provider/account/namespace/external identity becomes canonical. Repeating the identical identity is equivalent. The exact provider-capability version determines whether that identity is unique across operations; only account_namespace scope activates the partial unique index. A different verified identity never replaces the canonical binding: the service appends {action}_acceptance_conflict with stored status conflict, refreshes the projection to acceptance_conflict_requires_reconciliation, and inserts an M05 handoff outbox record.

7.4 Indeterminate and resend

A submission_indeterminate event is explicit, durable, and non-terminal. It creates an M06 reconciliation handoff; M02 does not retry the provider. A staff-authorized resend is a new operation with a new operation_id and resend_of_operation_id, attributable actor, controlled reason, and a warning that the original may have succeeded. It runs the full intent-before-action sequence.

8. Inbound observation and callback workflow

8.1 Observation transaction

- M03 verifies the webhook, resolves tenant/provider routing, claims the durable inbox row, normalizes the fact, and supplies trusted correlation according to the M11/M03 boundary.

2. M02 validates the observation contract, source event identity, producer authority, typed subject/source link, timestamp quality, and minimum-necessary metadata.
3. Within one M01 tenant transaction, the owning module applies the accepted domain state change and M02 appends one completed observation event plus links and any contract-required handoff. An operation projection is refreshed only when operation_id is present and the observation contract declares an operation projection rule; a null-operation observation creates no operation_projections row.
4. An equivalent source duplicate returns the prior committed result. A different fact with the same source identity is DEDUPE_CONFLICT; no history is overwritten.
5. M03 marks its inbox work complete only after the tenant transaction result is known; restart repeats safely.

8.2 Verified delivery/business outcomes

recordOutcome accepts facts only from an authorized M03 callback worker, M06 provider lookup, or owning-domain confirmation. Provenance must identify how the outcome was verified. Provider timestamps are used for semantic ordering only when source_time_quality is trusted. Missing or untrusted time cannot silently settle contradictory terminal evidence.

8.3 Terminal conflict

When incompatible verified terminal delivery or business facts exist, both remain authoritative. M02 appends {action}_delivery_conflict with stored status conflict, identifies the competing evidence, projects delivery_conflict_requires_reconciliation as non-terminal, and enqueues an M05 handoff. Only an authorized correction or manual-resolution event can resolve the conflict.

9. Deterministic projection design

Deterministic projection pipeline

The same complete event set must yield the same state, controlling evidence, and explanation.

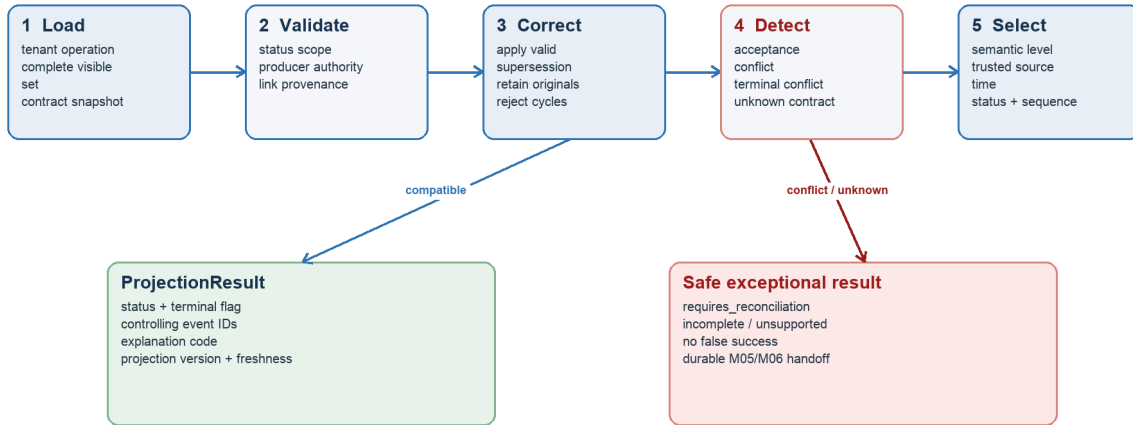


Figure 3. Versioned deterministic projection pipeline

9.1 Pure projection algorithm

1. Load the complete visible event set for one tenant operation and the exact referenced contract/status versions. Missing definitions yield `projection_unsupported`.
2. Verify immutable structure and partition equivalent duplicates. A stored duplicate should already be prevented; any inconsistent duplicate is an integrity defect.
3. Build the correction graph. Correction/resolution authorization is established at append time; the pure engine and rebuild validate stored graph and semantic references under the exact historical rules without rerunning current entitlement checks. A later-revoked entitlement does not automatically invalidate earlier stored correction/resolution evidence. Privileged-writer forgery remains outside this assurance. Apply valid corrections in `append_sequence` order under their exact historical rules; reject cycles, cross-operation targets, unsupported scope, or correction of correction when forbidden. Preserve all original facts for the timeline; incompatible overlapping resolutions remain conflict rather than latest-write-wins.
4. Detect canonical-acceptance violations before ordinary precedence. Any competing identity yields `acceptance_conflict_requires_reconciliation` until resolved by correction/manual evidence.
5. Detect incompatible verified terminal delivery/business outcomes before ordinary precedence. Untrusted/missing source time never erases a conflict.

6. If facts are compatible, choose the highest semantic level: delivery/business outcome over submission over intent. An observation with operation_id is folded only when its contract declares an operation projection rule. An observation with null operation_id produces no operation_projections row; its durable view is patient_events plus links and the subject timeline, while the owning module's accepted domain state changes atomically. Correction itself is not a business level.
7. Within a compatible level, compare trustworthy occurred_at, then contract status precedence, then append_sequence as the final total-order tiebreaker. received_at is diagnostic only and cannot move a terminal state backward.
8. Return status, terminality, flags, controlling and superseded event IDs, explanation code, projection_engine_version, contract versions, and source-set fingerprint.

9.2 Explanation codes

Code	Meaning
INTENT_ONLY	Committed request exists; no submission fact.
SUBMISSION_ACCEPTED	Canonical acceptance controls; delivery/business outcome absent.
SUBMISSION_REJECTED	Definitive submission rejection controls.
SUBMISSION_INDETERMINATE	Outcome unknown; M06 reconciliation required; non-terminal.
VERIFIED_OUTCOME	Compatible verified delivery/business evidence controls.
ACCEPTANCE_CONFLICT	Competing provider acceptance identity; M05 resolution required.
TERMINAL_OUTCOME_CONFLICT	Contradictory verified terminal outcomes; M05 resolution required.
CORRECTION_APPLIED	Authorized correction changes selected metadata or asserted-fact interpretation.
CONTRACT_UNSUPPORTED	Unknown/incompatible contract or status; success not claimed.
INTEGRITY_INCOMPLETE	Required source/link/contract evidence is missing or inconsistent.

9.3 Hybrid cache and rebuild

Every operation-scoped append transaction calls `m02_lock_operation(operation_id)` before dedupe resolution, event insertion, conflict detection, or projection work. The SECURITY DEFINER gateway performs `SELECT ... FOR UPDATE` on the exact operation_registry row; haloflow_runtime receives no UPDATE privilege and cannot issue the locking SELECT directly. The row lock survives return from the gateway and remains held by the surrounding TenantTransactionGateway transaction through COMMIT or ROLLBACK. After acquiring the lock under READ COMMITTED, the service loads the now-complete committed event set, appends the new fact, runs the pure engine, appends any required conflict event and outbox row, reruns the engine if generated evidence changed the set, and upserts operation_projections before COMMIT. Transactions touching multiple operations invoke the gateway

for operation_id values in ascending byte order. operation_registry is the lock target because its row always exists; a projection-row upsert is not safe.

This per-operation critical section makes concurrent contradictory facts converge immediately to one conflict projection and one effective M05 handoff. Operations remain independent, but the complete-set fold is $O(n)$ inside the lock; representative history-depth tests must satisfy OI-009, and any incremental optimization must produce byte-equivalent ProjectionResult and source fingerprint through the same engine.

An append with null operation_id is explicitly exempt from m02_lock_operation: there is no registry row and no cross-fact operation conflict to serialize. Source-event and producer-dedupe constraints protect equivalence. Such observation events remain authoritative in patient_events and patient_event_links, appear in the subject timeline, and do not create operation_projections; any current domain state belongs to the owning module's atomic state change.

The cache stores source_event_count, max append_sequence observed, source-set fingerprint, engine version, refreshed_at, and controlling IDs. The high sequence is a completeness diagnostic only, not a resume cursor. Reads may require cache version/freshness; when unsuitable, the service recomputes from events or returns explicit unavailable according to the consistency mode. A projection page preserves one selected engine version. Wrong-version or unsuitable items recompute only with existing elevated evidence authorization and budget, otherwise return per-item unavailable; fallback never persists or enqueues. The complete event/link/binding envelope is assembled in one SQL statement, so READ COMMITTED queries cannot be stitched across snapshots. Exact default/synthetic outputs and metadata paths follow the issued read manifest; unregistered clinical/support profiles are unavailable. Whole metadata-column reach at the read owner remains disclosed even though returned paths are restricted.

Pilot rebuild declares tenant_id, projection version, and a bounded operation/range selector, retains the exact ordered membership, and pins the engine and contract snapshot. Each operation runs in its own READ COMMITTED TenantTransactionGateway transaction: isolation check, operation lock, complete evidence load, pure fold, validated projection-only apply, COMMIT. Apply and the private finalizer also reject non-READ-COMMITTED execution before source comparison. Publication is per operation, not an atomic range swap or range-wide evidence snapshot. A mismatch under the held lock is an integrity/protocol error; the current operation rolls back and the run reports completed, failed and unattempted members. An interrupted run repeats the retained membership idempotently using new per-operation observations. Replay has no adapter or enqueue path; dry-run compares without applying. A signed run manifest records membership, pinned contract snapshot, per-operation source observations and truthful complete/partial/failed results.

10. Crash-durable control handoffs

10.1 Outbox record

Field	Rule
handoff_id	UUID primary key; stable for dispatcher retry.

Field	Rule
tenant_id	Trusted M01 context value copied structurally; never caller input.
operation_id / source_event_id	Tenant-bound evidence anchors; resolution revalidates against tenant schema.
handoff_kind	reconciliation, ordinary_failure, acceptance_conflict, delivery_conflict, event_signal, integrity_alert.
dedupe_key	Deterministic versioned key. M05: tenant + operation + kind. M06: tenant + operation. M11: tenant + source event + handoff kind + signal type. The CP0 contract baseline pins compact canonical UTF-8 encoding and the closed M02 emitted map: event_signal/m02.event.appended and integrity_alert/m02.integrity.incomplete, subject to consumer compatibility. Required-list completeness remains service correctness under the accepted finalization boundary.
payload	Versioned structural JSON only: controlled codes, pseudonymous tenant, IDs approved for internal use, no event metadata/error body.
state / attempts / available_at	pending, leased, delivered, dead_letter; bounded backoff; mutable operational state.
lease_owner / lease_expires_at	Fenced dispatcher claim; expired leases are reclaimable.
created_at / delivered_at	DB timestamps for commit-to-publication and backlog-age objectives.

10.2 Delivery semantics

1. The source event and required outbox row commit atomically in the tenant transaction. No in-process-only trigger satisfies the contract.
2. Dispatchers enumerate tenants through a paged M01-bound scheduler in fair round-robin order, claim a bounded number of rows per tenant with SKIP LOCKED/fencing, and deliver a versioned command. Tenant enumeration and row claims are separately bounded so one tenant cannot monopolize a cycle.
3. M05/M06 persist or return the effective control record under their deterministic tenant-scoped unique key before acknowledging.
4. After acknowledgement, the dispatcher marks the outbox row delivered. A crash before that update repeats delivery and converges at the consumer.
5. M11 signal delivery is non-authoritative; lag alerts do not change the event or repeat the external side effect.
6. Dead-letter state preserves structural evidence and pages the owning module; it does not mutate the source event or falsely close a case.
7. Metrics record tenant count, tenants visited, full-sweep duration, oldest pending age, rows claimed, empty-query ratio, and commit-to-publication latency. Architecture review is triggered when tenant growth or sweep duration breaches an OI-009 objective; any shared pending-tenant hint requires M01 classification, non-authoritative repair logic, and architecture approval.

11. Query, pagination, and authorization

11.1 Operation projection reads

getOperationProjection accepts only an operation_id inside an authorized TenantContext. M01 and the service validate operation/subject capability and any requested support or clinical view capability before dispatch through the registered statement boundary. The service enforces the requested consistency mode; the selected read view or bounded read function applies the reviewed field allowlist and local-row constraints. No provider/event metadata is returned without the matching approved view capability. A missing operation uses non-enumerable behavior consistent with M01.

11.2 Bulk projection

Selector	Bound	Stable order / cursor
operation_ids	Maximum 200 IDs per request baseline	Caller order echoed with per-item result; one tenant only.
batch/run link	Maximum 500 results/page baseline	business link + operation_id; opaque cursor binds tenant, selector hash, sort key, projection version.
approved queue selector	Maximum 200 results/page baseline	queue sort key + operation_id; queue authorization checked by owner.

Numeric bounds are conservative implementation defaults pending M02-OI-009 production-like testing. Every page declares generated_at, projection_engine_version, freshness policy, and per-item errors. A cursor is signed, expires, cannot change tenant/selector, and does not expose schema or PHI. The CPO baseline additionally pins provisional m02_limits_v1 hard maxima, including a 1,000-event/32MiB complete fold and 1MiB completion envelope. Exceeding full-history or completion capacity fails explicitly and rolls back the append; repeated callbacks may fail until controlled remediation. No prefix fold, truncation or production-capacity guarantee is permitted. SignerProvider signs the canonical envelope together with the service-derived purpose binding, which is not transmitted in client cursor envelopes; verification reconstructs the binding and rejects tampering, wrong purpose, expiry or a mismatched request binding before querying. A non-production fixture provider is rejected in production; business-key HMAC material is never reused.

11.3 Subject timeline

Timeline ordering is occurred_at when trusted, otherwise created_at, with append_sequence as the stable final key. This display order does not replace projection conflict rules. The default view exposes event type, normalized status, structural timestamps, correction relationship, and masked provenance; metadata fields require a contract-specific view capability and are filtered field by field.

11.4 Stale-intent evidence query

listStaleIntents is the M02-owned evidence contract used by the M06-owned scheduled detector. Within one authorized tenant, it anti-joins committed intent events against every qualifying submission

fact, excludes already resolved or superseded candidates, and returns only rows whose created_at is older than the caller's approved cutoff. It reads authoritative events rather than projection cache state. Discovery never changes projection and never authorizes a provider resend.

The baseline page limit is 200 pending OI-009. Stable order is requested_event.created_at then operation_id; the signed cursor binds tenant, cutoff, action selector, contract snapshot, and page boundary. M06 revalidates the candidate and performs provider lookup when supported, then uses recordSubmissionOutcome with a deterministic stale-detector producer key to append submission_indeterminate and its M06 outbox row idempotently when outcome remains unknown.

12. Corrections, manual resolution, and privileged evidence

Action	Required controls	Projection effect
Metadata correction	Correction capability, actor/workload, reason, corrected_event_id, replacement allowlisted fields, audit record.	Original fact remains; selected metadata view is superseded.
Asserted-fact correction	Enhanced capability, evidence source, reason, affected assertion, two-person approval where policy requires.	Projection excludes/supersedes the corrected assertion according to contract.
Manual resolution	M05 case, actor, evidence source, controlled resolution path, event link, audit. The synthetic profile does not cover confirmed_no_side_effect for an indeterminate-only history without an existing rejected fact. A versioned owning-module rule is required before releasing that affected FR-035 workflow; do not fabricate provider rejection, failure or test coverage. Canonical acceptance-conflict resolution may select the canonical accepted fact under the approved provider-lookup rule; selecting a noncanonical identity is unsupported/INTEGRITY_INCOMPLETE and never replaces the binding.	Appended resolution event becomes controlling only under versioned rule.
Authorized resend	Explicit warning/acknowledgement, actor, reason, original operation, owning-module new business key.	New operation_id linked by resend_of_operation_id; original remains indeterminate/history.
Export/disposition	Purpose-specific identity, legal-hold check, approved scope, completion evidence, access audit.	No ordinary projection change; archival access behavior follows policy.

Integrity rule. No privileged workflow edits patient_events in place. If policy authorizes physical archival or lawful destruction, it must preserve attributable disposition/tombstone evidence and cannot use runtime credentials.

13. Security and privacy design

13.1 Roles and grants

Role / identity	Allowed	Explicitly denied
haloflow_runtime	Execute approved M02 procedures, including the two lock gateways; read projections/timelines/stale-intent pages by tenant capability; request tenant key operations through KeyProvider.	Raw table INSERT/UPDATE/DELETE/TRUNCATE; direct SELECT FOR UPDATE/KEY SHARE; direct key material; reference administration; correction/export/disposition.
haloflow_m02_lock_owner	NOLOGIN owner of the v1 m02_lock_operation and m02_begin_key_scope gateways; acquire exact-row locks and return minimum structural values under a fixed safe search_path. V1 installs no m02_rotate_key_version function or callable statement; its design and authority are deferred to OI-009.	Application login; provider calls; dynamic SQL or caller schema; broad data return; ordinary event mutation; PUBLIC EXECUTE.
haloflow_m02_append_owner	Own ordinary begin/submission/outcome/observation gateways; read required tenant evidence and perform reviewed append mutations; invoke key-scope and operation-lock gateways. Own the append-completion wrapper; invoke the projection-owner operation finalizer and private event-handoff route through reviewed internal EXECUTE grants.	Direct key-control SELECT; correction/resolution authority; DELETE/TRUNCATE; caller-controlled SQL/schema; provider I/O.
haloflow_m02_correction_owner	Own restricted correction/resolution gateways; append replacement evidence and required audit/handoff records through reviewed paths; invoke operation-lock gateway. Own the correction-completion wrapper; invoke the projection-owner operation finalizer and private event-handoff route; insert only reviewed structural audit columns into the existing tenant access_audit_outbox using trusted service-derived attribution.	Rewrite original event or canonical acceptance binding; direct key-control SELECT; DELETE/TRUNCATE; unaudited correction.
haloflow_m02_read_owner	Own tier-specific projection/timeline views and bounded read functions; SELECT only required underlying evidence/derived fields. Own the internal operation-evidence read route for authorized runtime, correction and projection-worker callers; expose only manifest-enumerated minimum-necessary engine inputs under distinct internal capability checks, never as public tier output.	Base-table mutation, outbox delivery mutation, key-control access, provider I/O. View ownership is administrative authority over those views, not a claim that the owner has only SELECT on its own objects.

Role / identity	Allowed	Explicitly denied
haloflow_m02_projection_owner	Own bounded projection rebuild/finalization gateways; read required evidence and mutate derived projection columns; invoke operation-lock gateway. Own the private event-handoff route for null-operation evidence; accept internal finalization/handoff calls from append and correction owners through reviewed EXECUTE grants, with no direct public runtime EXECUTE on those private functions. Own the private projection-only helper used by rebuild apply; replay cannot enqueue. Read only the approved four shared.tenants routing columns for fixed-schema tenant binding; cross-row routing-metadata reach is disclosed.	Event/binding mutation, key-control access, provider I/O, new downstream delivery during replay.
haloflow_m02_outbox_owner	Own outbox lease/finish gateways; read structural outbox records and update reviewed delivery/lease columns under fencing. Read only tenant_id, schema_key, lifecycle_state and schema_version from shared.tenants for fixed-schema tenant-binding checks; the column grant exposes routing metadata across registry rows, not event metadata. M01 retains schema-version compatibility checks. This is the approved O1 control-plane routing exception; unrelated tenant event, clinical and outbox data remain denied.	Event metadata/evidence mutation, key-control access, unrelated tenant data, provider side effects.
haloflow_key_provider	V1 verifies configured-provider fail-closed behavior and version lookup continuity where applicable. Cloud KMS, producer-registration and rotation-specific behavior in the following requirement is deferred to OI-009: Read approved tenant/version key envelopes and invoke Cloud KMS decrypt/derive for the requested tenant scope.	Database mutation, provider calls, cross-tenant bulk key access, key export, or logging plaintext key material.
haloflow_projection_worker	Read tenant events and upsert derived projection rows through bounded rebuild procedure. Invoke m02_lock_operation through the distinct m02.projection.rebuild.lock capability and direct EXECUTE grant; apply only the projection result through the no-enqueue rebuild route. No owner-role membership.	Provider calls, event mutation, cross-tenant query in one transaction.
haloflow_handoff_dispatcher	Lease/update outbox operational fields and deliver structural commands/signals.	Event metadata access beyond required structural fields; source event mutation.
haloflow_correction	Append approved correction/manual-resolution event through restricted gateway; write privileged audit evidence. Invoke m02_lock_operation through the distinct m02.correction.lock capability and direct EXECUTE grant before evidence loading and correction work. No owner-role membership.	Update/delete originals; unscoped tenant switching; correction without reason/evidence.
haloflow_event_disposition	Approved archival/export/destruction under legal-hold and audit controls.	Ordinary runtime use or silent evidence removal.
haloflow_migrator	Install schema, references, contracts, indexes, grants, and compatibility checks.	Business workflow execution or support use.

13.2 Threat controls

Threat / failure mode	Prevention	Detection / containment
Cross-tenant event/link	M01 context; tenant-local repositories; verified link resolvers; no schema argument.	Negative tests, security alert, non-enumerable response.
Side effect without intent	Adapter contract requires committed OperationStart; owner code rule and fault injection.	Stale provider registry/reconciliation anomaly and release-blocking tests.
Duplicate side effect	Stable operation_id as provider key; operation business fingerprint; owner retry contract.	Acceptance conflict metric, provider lookup, staff review.
Evidence mutation	Append-only grants; separated privileged identities; database auditing.	Integrity reconciliation, privileged audit, immutable backup/export evidence.
Lock privilege escalation	Runtime uses approved EXECUTE gateways and SELECT-only tier views; dedicated NOLOGIN function owners, exact safe search_path, invariant tenant-object-unqualified exact-row function bodies, minimum structural returns, and PUBLIC EXECUTE revocation constrain the gateway paths. Installation SQL uses validated tenant-schema qualification. Projection-owner and outbox-owner binding checks use the provisioning-fixed schema to resolve the active shared.tenants row and require equality with app.tenant_id; this is routing consistency, not principal authentication. Their expressly approved four-column registry read is the control-plane routing exception; unrelated tenant event/clinical data remains denied. Shared grants are issued by the owner-run Alembic role revision, not tenant units. M02 verifies both owners and exact column privileges; M01 TC-E22 table-level assertions alone do not cover this extension. The provisioning-rendered schema binding constant is the only permitted per-tenant body difference for projection-owner and outbox-owner binding checks, verified against the trusted per-tenant rendered expected body.	Grant-manifest diff, procedure checksum, direct-lock denial test, and audit of definer changes.
Forged producer/source	Workload identity and capability; source namespace contract; M03 verification boundary.	Producer/status mismatch alert; quarantine unsupported fact.
PHI leakage	Metadata JSON Schema/size; field allowlists; masked views; structural-only outbox/log signal.	Automated prohibited-field scanning and privacy review.
Projection deception	Pure versioned engine; source fingerprint; unknown/conflict never success; cache rebuildable.	Live-vs-rebuild comparison, explanation and version metrics.
Correction abuse	Restricted capability, attribution, evidence, reason, optional dual approval.	Volume/outlier alert and retrospective review.
Replay side effect	Replay runtime has no adapter capability; dry-run/idempotent handoff compare.	Provider-call canary assertion and replay audit.
Business-key key loss/rotation	Database-controlled active version, retained key envelopes, bounded tenant/version cache, protected backup and restore.	V1 verifies configured-provider fail-closed behavior and version lookup continuity where applicable. Cloud KMS, producer-registration and rotation-specific behavior in the following requirement is deferred to OI-009: Fail-closed beginOperation, key-availability alert, rotation canary, lookup-continuity test.

13.3 Business-key HMAC custody and lifecycle

1. OI-009 follow-up requirement; not a v1 delivery or activation precondition under D-11.10.
KeyProvider obtains tenant/version-specific HMAC key material from a Cloud KMS-protected envelope under a dedicated workload identity. The database stores only key version and control metadata; key bytes, raw business keys, and fingerprints never enter logs or central telemetry. V1 ships the KeyProvider abstraction, HMAC, version controls and provenance, with LocalDevKeyProvider restricted to non-production; production fails closed without a production-capable provider. An unkeyed hash is not a substitute.
2. OI-009 follow-up requirement; not a v1 delivery or activation precondition under D-11.10.
Decrypted keys may be cached in process for at most 15 minutes, keyed by tenant_id plus version, zeroized on eviction, excluded from crash dumps, and invalidated on rotation notification. A cache miss or expired entry requires successful KMS access; otherwise beginOperation fails closed before any provider call.
3. OI-009 follow-up requirement; not a v1 delivery or activation precondition under D-11.10.
Rotation uses expand -> register target -> verify live producer compatibility -> atomic active-version flip -> retained lookup -> contract. Each authorized producer registers its workload identity, deployment generation, supported key versions, and expiring heartbeat at startup. m02_rotate_key_version refuses the flip unless the deployment controller has attested completion of the target generation and every live required producer registration supports the target; stale, missing, or unsupported registrations fail the gate and alert. m02_begin_key_scope holds FOR KEY SHARE on the control row while rotation holds FOR UPDATE, so the flip waits for in-flight writers and no old-version write can race a new-version write.
4. OI-009 follow-up requirement; not a v1 delivery or activation precondition under D-11.10.
Retained key envelopes remain recoverable for at least the maximum operation lifetime and the approved backup window. Deletion requires evidence that no live or recoverable operation depends on the version, legal/retention approval, and a restore drill. If a required key is lost or cannot be restored, operation acquisition stops; the system never silently substitutes a new key or creates replacement operation IDs.

13.4 Minimum-necessary metadata

1. Prohibited in generic metadata and central signals: raw provider payloads, message/fax content, full phone numbers, names, dates of birth, diagnosis data, secrets, tokens, unrestricted URLs, raw SQL, stack traces, and free-text error bodies.
2. Permitted only by an approved event contract: controlled enums, bounded numeric measures, last-four digits where explicitly justified, sanitized provider code, masked resource class, and structural version/provenance data.
3. Full sanitized operational detail needed for tenant support remains in tenant Cloud SQL under contract-specific fields and view permissions, not in Cloud Logging.

4. Every shared M02 column is registered in the M01 machine-readable classification manifest; CI rejects missing classification or PHI without an approved exception.

13.5 Retention-locked export evidence

Under the approved D-09 v1 baseline, immutable export evidence means an encrypted Cloud Storage export written under a retention-locked bucket policy with object versioning and access logging. Each export has a separately stored signed manifest containing tenant scope, declared event range, schema/contract versions, row counts, canonical file digests, export time, producing identity, legal-hold state, and verification result; the manifest contains no event payload or PHI. Restore and digest verification are exercised before pilot; a control failure reopens OI-010. FR-026 policy still controls archival and lawful disposition.

14. Failure behavior and error taxonomy

Condition	Caller behavior	Durable / operational action
Intent commit fails	No OperationStart success; adapter must not run.	Rollback; sanitized append failure signal if possible.
Provider outcome unknown	Return/record indeterminate, never failed/success.	Append submission_indeterminate; enqueue M06 reconciliation.
Outcome append fails after acceptance	Do not claim recorded outcome.	Committed intent remains; stale detector/provider lookup recovers.
HMAC key unavailable or unsupported	beginOperation fails closed before intent/provider execution.	Use an unexpired tenant/version cache entry or restore approved key service; alert and never substitute a fingerprint.
DB commit uncertain	APPEND_COMMIT_UNKNOWN with event/dedupe lookup token.	Resolve prior commit before retry; never invent replacement event_id.
Equivalent duplicate	Return prior committed AppendResult.	Increment duplicate_suppressed metric; no second event/handoff.
Dedupe content mismatch	Generic unavailable/invalid result.	Reject, quarantine structural attempt evidence, security/integrity alert.
Acceptance/terminal conflict	Return non-terminal requires_reconciliation projection.	Retain facts; enqueue deterministic M05 handoff.
Unknown contract/status/provider mapping	Explicit unsupported; never success.	Quarantine/alert compatibility owner; no ordinary projection.
Projection cache unavailable/stale	Recompute if policy/budget allows, else explicit unavailable with freshness.	Alert on lag; authoritative events unchanged.
Handoff delivery fails	Source append remains successful.	Retry leased outbox; consumer dedupe; dead-letter and alert after budget.
Cloud Logging unavailable	No rollback or provider replay.	Outbox signal remains pending; M11 lag alert.
Unauthorized correction/cross-tenant link	Non-enumerable denial.	No event change; security audit/alert.

15. Observability and service objectives

Signal	Minimum dimensions	Alert intent
Operation acquisition	outcome, action_family, owner, created_or_existing, service/version	Unexpected collision or acquisition failure.
Event append	event level/type/status, result, producer, contract version, latency	Sustained failure, invalid contract, commit ambiguity.
Duplicate/conflict	dedupe class, conflict kind, action family, producer	Mismatch, acceptance conflict, terminal conflict growth.
Projection	engine version, explanation, freshness, source count, compute latency	Unsupported contract, live/rebuild mismatch, lag.
Handoff	kind, state, attempt, backlog age, consumer, latency, tenant sweep	M05/M06/M11 delivery breach, sweep-bound breach, or dead letter.
Stale/indeterminate	age bucket, action family, owner, reconciliation state	Growing unknown-outcome backlog.
Key service	V1 verifies configured-provider fail-closed behavior and version lookup continuity where applicable. Cloud KMS, producer-registration and rotation-specific behavior in the following requirement is deferred to OI-009: KMS result, decrypt latency, cache state, producer generation, target/active version, rotation gate result	Cold-start acquisition failure, expiring cache headroom, unsupported producer, or KMS objective breach.
Correction/privileged	reason/path code, actor class, result, approval class	Abuse, unexpected volume, unauthorized attempt.
Fairness	tenant pseudonym, workload, rate, page size, replay range, throttling	One tenant threatens peer objective or unbounded work.
Integrity	check kind, mismatch class, sequence anomaly class, deployment	Evidence/cache/outbox inconsistency; never alert on a gap alone.

Pending objectives. Append latency, projection latency, full-range replay duration/event bound, handoff backlog age, signal publication latency, page sizes, fairness thresholds, Cloud KMS decrypt/error objectives, cold-start key availability, and bounded-cache headroom require M02-OI-009 approval. Instrumentation ships before values are enforced.

16. Reliability, performance, and capacity

Concern	Design mechanism	Verification
Durability	Synchronous Cloud SQL commit defines append success; backups and point-in-time recovery protect source evidence. Cloud KMS availability is a hard dependency for uncached beginOperation and therefore every new external side effect on a cold process.	Fault injection at begin/append/commit, Cloud KMS outage with warm and cold processes, and database/key restore exercises.
Concurrency	operation_registry row lock per operation, deterministic multi-operation lock order, unique constraints, content fingerprints, canonical binding, SKIP LOCKED outbox leases.	Barrier-synchronized contradictory appends produce conflict evidence/projection and one effective handoff; deadlock regression.

Concern	Design mechanism	Verification
Query cost	Operation/link indexes, cached projection, bounded bulk APIs, stable pagination, no routine JSON scan.	Representative deep timelines, batches, queues, explain plans.
Rebuild	Retained bounded membership; per-operation READ COMMITTED lock-before-load transactions, pure engine, source fingerprints, signed observation manifest, full-membership retry after interruption, provider-disabled and enqueue-disabled runtime.	Live/rebuild equivalence and interruption restart.
Fairness	Paged round-robin tenant sweep plus per-tenant rate, concurrency, page, batch, outbox, and replay bounds with backpressure.	Saturate one tenant and grow tenant count while measuring peer and full-sweep objectives.
Compatibility	Versioned contracts/engine, database-controlled key write version, live producer capability registrations, deployment-generation attestation, expand-contract migrations, startup matrix, and old fixtures retained.	OI-009 follow-up requirement; not a v1 delivery or activation precondition under D-11.10. Rolling deployment and key-rotation race tests; refuse activation for stale, missing, or unsupported producer registrations.
Recovery	Stable IDs, explicit indeterminate state, recoverable outbox, recomputable cache, retained key envelopes, and exercised KMS/key restore.	Crash at every sequence boundary; cold-process KMS outage and key-restore drill; no false success, substitute fingerprint, or automatic resend.

17. Verification strategy and release gates

Test layer	Required coverage	Release evidence
Unit / contract	Contract/status scope, producer authority, metadata schemas, fingerprints, correction graph, explanation codes.	Deterministic positive/negative fixtures for every core contract.
Database	Append-only grants, SECURITY DEFINER lock gateways, safe search_path/PUBLIC revocation, FK/link checks, dedupe uniqueness, canonical binding, and projection/outbox transactionality.	PostgreSQL 17 integration results, procedure checksums, and permissions manifest proving runtime has no raw UPDATE.
Projection	Shuffled arrival, equal time/rank, untrusted time, corrections, acceptance and terminal conflicts, unknown versions.	Same result, controlling IDs, explanation, and version for same set.
Concurrency	Barrier-synchronized equivalent and contradictory appends through m02_lock_operation; key-version flip versus m02_begin_key_scope; callbacks, corrections, outbox claims, and commit ambiguity.	Gateway-acquired locks persist through the caller transaction; all facts retained; immediate conflict event/projection; one M05 item; no operation fork, duplicate send, or deadlock.
Crash recovery	Before/after intent, provider call, outcome append, event/outbox commit, consumer ack, delivered mark.	No call without intent; recoverable handoff; no M02 resend.
Security/privacy	Forged tenant/subject/producer, metadata injection, direct locking SELECT, unsafe search_path, PUBLIC procedure access, prohibited grants, correction abuse, and telemetry inspection.	Security/privacy sign-off; zero open critical/high; PHI-safe outputs; definer gateways cannot be repurposed for mutation.
Replay/integrity	Clean and interrupted full-range rebuild, out-of-order commits, sequence gaps, privileged mutation simulation.	Live/rebuild equality; gap false positives absent; attributable alerts.

Test layer	Required coverage	Release evidence
Performance/fairness	Append under per-operation lock, projection, stale-intent anti-join using idx_patient_events_stale_intent_scan, patient/link explain plans, bulk pages, tenant sweep, outbox backlog, rebuild, and KMS dependency behavior.	Approved OI-009 objectives, index plan, KMS cold/warm objectives, review triggers, and peer-tenant protection.
Module contract	M03 observation/callback, M05 work handoff, M06 reconcile/bulk, M10 timeline, M11 signals.	Versioned consumer/provider contract tests.
E2E	Two tenants: accepted, competing acceptance, rejected, indeterminate, delivered, terminal conflict, observation, correction, resend.	Isolation assertions and clinical/operational owner acceptance.

17.1 Mandatory negative scenarios

1. Attempt adapter execution after failed/uncommitted beginOperation; prove the provider stub receives no call.
2. Reuse one business key with different action, subject, or owner; reject instead of returning an unrelated operation.
3. Submit same source event concurrently with equivalent and non-equivalent bodies; collapse only equivalent attempts.
4. Start two real concurrent transactions behind a barrier, append contradictory terminal outcomes to one operation, and prove the second lock holder sees both facts, appends one conflict event/outbox row, and commits the non-terminal conflict projection.
5. As haloflow_runtime, prove direct SELECT FOR UPDATE/KEY SHARE and every raw UPDATE are denied; call each SECURITY DEFINER lock gateway and prove the exact row remains locked after gateway return until the surrounding transaction ends. Reject unsafe search_path, PUBLIC EXECUTE, dynamic SQL, wrong-tenant rows, and missing rows.
6. Race a business-key active-version flip against old/new producer acquisitions; prove the control-row lock permits one write version, retained lookup returns one operation_id, and an unsupported producer fails closed.
7. OI-009 follow-up requirement; not a v1 delivery or activation precondition under D-11.10. Attempt key activation with a stale heartbeat, missing required producer, unsupported target version, or incomplete deployment-generation attestation; m02_rotate_key_version refuses the flip and emits a structural alert.
8. Create competing provider acceptance identities under both unique and non-unique capability scopes; retain all facts, enforce only the declared namespace scope, and project non-terminal conflict where required.
9. Attempt to COMMIT patient_events with null patient_id and no approved link, including direct privileged test inserts; prove trg_patient_event_subject_required rejects the transaction.
10. Append a valid observation with null operation_id; prove no registry lock is sought, source dedupe remains effective, no operation_projections row is created, the subject timeline contains the fact, and the owning domain state commits atomically.

11. Run `listStaleIntents` at representative depth and cutoff selectivity; require an explain plan using `idx_patient_events_stale_intent_scan` and the operation submission index without an unbounded tenant scan.
12. Start cold and warm processes during Cloud KMS latency, denial, and outage faults; verify bounded cache behavior, fail-closed acquisition, objective signals, and no substitute key or provider call.
13. Shuffle `received_at`, `occurred_at`, `append_sequence` allocation, and commit order; projection remains set-deterministic and never uses sequence as a resume cursor.
14. Forge tenant, subject/source link, causation, correction target, operation, provider namespace, actor, producer, or public correlation ID.
15. Insert `projection_only` status into `patient_events`, unknown contract version, oversized/prohibited metadata, or unrestricted failure text.
16. Crash between source event and handoff dispatch, consumer persistence and acknowledgement, and acknowledgement and delivered mark; converge to one effective record.
17. Run replay with adapter canaries; prove no provider call and no duplicate effective M05/M06 work.
18. Simulate legitimate rolled-back sequence gaps and privileged mutation; alert only on attributable integrity violations, not the gap alone.

18. Migration, rollout, and rollback

Phase	Entry / exit intent
1. Architecture	Approve ADR-011 scope and design decisions D-02–D-09; assign owners and provider/privacy/SRE gates.
2. Shared references	Add inactive type/status/contract/provider-capability versions, producer-key capability registration, and classification inventory; create the next HMAC version as lookup-only; deploy readers that tolerate inactive additions.
3. Tenant expand	Create operation/key-control, event/link, binding, projection and outbox tables; add <code>idx_patient_events_stale_intent_scan</code> , integrity triggers, separate approved execution owners, the two v1 lock gateways, exact safe function <code>search_path</code> , PUBLIC revocation, and reviewed runtime gateway/view grants in synthetic tenants. V1 installs no rotation gateway; rotation execution and heartbeat machinery remain OI-009 follow-up.
4. Library	Ship versioned services/engine, the v1 KeyProvider and HMAC/version-provenance path, static ownership and direct-lock checks, adapter-disabled replay tooling, and compatibility validation. Producer startup/heartbeat registration and rotation orchestration remain OI-009 follow-up under D-11.10.
5. Synthetic validation	Pass the v1 two-tenant isolation, definer-grant/search-path, lock-persistence, subject-trigger, concurrent-conflict, null-operation observation, key-unavailability fail-closed, crash, correction, replay, indexed stale-query, tenant-sweep and PHI-safe telemetry suites. Producer-gated rotation and Cloud KMS-specific outage/integration suites are OI-009 follow-up obligations and are not claimed as v1 evidence.
6. Canary producer	Enable one external side-effect and one observation path; shadow projection and compare every explanation.
7. Consumer canary	Enable M05/M06 handoffs and M10/M11 reads/signals behind version gates; monitor backlog, duplicate suppression, and lag.

Phase	Entry / exit intent
8. Limited pilot / GA	Pilot only after provider, key restore, retention/export, privileged-integrity, runbook, and SLO gates; broaden after exercised recovery and tenant-sweep review.

18.1 Rollback

Rollback disables incompatible producers/consumers, preserves every appended event, restores the last compatible contract and ProjectionEngine version, quarantines unknown types, drains or pauses dispatch, and rebuilds derived state when required. New append-only columns/tables remain under expand-contract until every deployed reader is compatible. A migration that cannot reverse safely uses a forward fix and dual-read window; rollback never deletes newer events or resends providers.

19. Open decisions and prerequisites

ID	Decision	Design status	Owner	Required by
M02-OI-001	ADR-011 and D4 mechanical status stamps	Open external gate	Architecture + Engineering	Before schema freeze
M02-OI-002	Physical level model and tables	Resolved in draft D-02/D-03	Data Architecture	Approve with design
M02-OI-003	Producer dedupe constraints	Resolved in draft D-06	Platform + Data	Validate in DB tests
M02-OI-004	Projection storage	Resolved in draft D-04	Architecture + Platform	Approve with design
M02-OI-005	Notifyre/provider capabilities	Open production blocker	Integration + Product	Before automated takeover
M02-OI-006	Retention/legal hold/disposition/correction policy	Open production blocker	Privacy + Compliance + Security	Before pilot
M02-OI-007	Core seed/extension catalog and compatibility process	Open content freeze	Product + Engineering	Before seed migration
M02-OI-008	Crash-durable handoff mechanism	Resolved in draft D-05	Architecture + M05 + M06	Approve before M06 freeze
M02-OI-009	Measured objectives, bounds, and critical dependency budgets	Open readiness gate; includes Cloud KMS	Product + SRE	Before production readiness
M02-OI-010	Cryptographic tamper evidence	Resolved for v1 by D-09: no hash chain; signed retention-locked exports	Security + Privacy + Compliance	Approved with design; revalidate before pilot

20. Requirement traceability

Requirement	Design coverage	Primary evidence
M02-FR-001	operation_registry UUID identity	Acquire/reuse uniqueness tests

Requirement	Design coverage	Primary evidence
M02-FR-002	beginOperation atomic registry + requested append	Pre-commit fault injection; provider call spy
M02-FR-003	Business fingerprint, definer-controlled active-version gate, live producer compatibility, and stable registry row	V1 verifies configured-provider fail-closed behavior and version lookup continuity where applicable. Cloud KMS, producer-registration and rotation-specific behavior in the following requirement is deferred to OI-009: Restart/takeover, producer-registration denial, and key-rotation race
M02-FR-004	Adapter idempotency contract; Notifire direct UUID	Byte-for-byte adapter tests
M02-FR-005	event_id and source/producer dedupe	Concurrent equivalent/distinct appends
M02-FR-006	Intent contract validation	External ID/status negative schema tests
M02-FR-007	Normalized submission result mapping	Accepted/rejected/timeout/unknown fixtures
M02-FR-008	Verified recordOutcome provenance	Separate lifecycle timeline
M02-FR-009	recordObservation plus atomic state callback	Duplicate observation/state transaction
M02-FR-010	Append-only tables, EXECUTE-only runtime, and SECURITY DEFINER lock gateways	Raw UPDATE/locking SELECT denial plus gateway lock-persistence test
M02-FR-011	Two dedupe forms plus content comparison	Retry/concurrency/conflict retention
M02-FR-012	Nullable patient_id, typed links, deferred subject constraint	Missing/cross-tenant link COMMIT rejection
M02-FR-013	Three timestamps and time-quality code	Out-of-order/untrusted-time fixtures
M02-FR-014	Pure ProjectionEngine algorithm	Ordered/shuffled/equal-rank fixtures
M02-FR-015	Serialized append plus delivery_conflict event/projection/outbox	Concurrent conflict retention and one M05 item
M02-FR-016	Authoritative stale-intent query, named partial scan index, and explicit indeterminate event/M06 outbox	Indexed anti-join, idempotent detector, no resend
M02-FR-017	Correction graph and privileged append	Original + correction + explanation
M02-FR-018	Shared status scope/reference validation	Projection-only insert rejection
M02-FR-019	Versioned EventContract registry	Historical fixture compatibility
M02-FR-020	JSON Schema, allowlist, 8 KiB bound	Prohibited field/size tests
M02-FR-021	Correlation field separated from operation/link	Cross-request workflow trace
M02-FR-022	Versioned provider namespace and capability-scoped uniqueness	Reused external ID across providers/accounts/scopes
M02-FR-023	Normalized adapter outcome	Every raw/unknown status contract test
M02-FR-024	M01-bound repositories	Forged tenant/schema/subject tests
M02-FR-025	Structural M11 outbox signal	Logging outage and PHI scan
M02-FR-026	Purpose-specific disposition workflow	Legal-hold/grant/evidence tests
M02-FR-027	Trusted tenant_id and verified shared links	Forged binding/resolution tests

Requirement	Design coverage	Primary evidence
M02-FR-028	Transactional outbox plus M02 stale-intent evidence query	Crash/retry/detector/consumer-dedupe tests
M02-FR-029	operation_acceptance_bindings + conflict path	Concurrent equivalent/competing identity
M02-FR-030	ordinary_failure M05 handoff	Replay to one effective work item
M02-FR-031	M11/M03 correlation trust contract	Public injection/internal propagation
M02-FR-032	M01 classification and shared inventory CI	Unclassified/PHI exception failures
M02-FR-033	Bounded listOperationProjections	No N+1/unbounded/cross-tenant result
M02-FR-034	Bounded tenant sweep and outbox commit-to-signal metric	Sweep/latency breach without rollback/replay
M02-FR-035	Manual resolution and linked new resend	Distinct attributable operation and warning
M02-NFR-001	Cloud SQL commit defines success	Commit/log-outage fault injection
M02-NFR-002	Fail closed before required intent, including cold-process Cloud KMS dependency	Warm/cold dependency outage behavior; no provider call
M02-NFR-003	Definer-gateway per-operation locks, constraints, pure engine, fenced outbox	Privilege denial, lock persistence, barrier-synchronized conflict, and deadlock regression
M02-NFR-004	Hybrid projection and indexes	Representative latency/query-cost tests
M02-NFR-005	Versioned contracts/engine plus database-controlled key version and live producer compatibility gate	V1 verifies configured-provider fail-closed behavior and version lookup continuity where applicable. Cloud KMS, producer-registration and rotation-specific behavior in the following requirement is deferred to OI-009: Rolling deployment registration/key rotation/historical replay
M02-NFR-006	PHI-safe signal catalog	Metric/dashboard verification
M02-NFR-007	Owned failure/conflict/lag/abuse alerts	Alert routing exercises
M02-NFR-008	Bounded full-range replay	Clean/interrupted equality; no side effects
M02-NFR-009	Paged tenant sweep, bounds, pagination, backpressure	Tenant-growth/noisy-neighbor objective test
M02-NFR-010	Privileged roles, audit, integrity verifier, signed locked exports	Mutation, restore/digest, gap false-positive tests

Appendix A. Repository and implementation controls

1. Only the M02 package exports OperationService, EventAppendService, ContractRegistry, ProjectionEngine, repositories, and handoff codecs; modules consume typed interfaces.
2. Static checks reject direct patient_events mutation outside M02 migrations/tests, any runtime raw UPDATE or direct SELECT FOR UPDATE/KEY SHARE on M02 tables, bypass of m02_lock_operation/m02_begin_key_scope, provider calls without a committed OperationStart, caller schema parameters, untyped metadata dicts, projection status literals outside generated references, and bypass of the active business-key version contract.

3. Event and handoff schemas generate typed producer/consumer fixtures from the same versioned contract source; generated code is reviewed and pinned by checksum.
4. The v1 change set includes reference seeds, contract/provider-capability rows, retained version/provenance schema, patient subject trigger, key-version controls, idx_patient_events_stale_intent_scan, approved NOLOGIN execution owners, the two tenant lock gateways with invariant tenant-object-unqualified bodies and exact safe search_path, PUBLIC revocation, runtime gateway/view grants, classification entries, startup compatibility declarations, and rollback/forward-fix notes. Installation object references use validated tenant-schema qualification. Producer heartbeat machinery and rotation function installation/execution remain deferred to OI-009 under D-11.10.
5. Replay tooling runs without provider-adapter or enqueue capability and emits a signed run manifest containing tenant, exact retained membership, the pinned contract snapshot, engine and limits-profile versions, per-operation observed source counts/fingerprints and dispositions, aggregate counts, elapsed time and complete/partial/failed result. These are per-operation evidence observations, not a range-wide database snapshot. Signing or persistence failure after any operation commit is reported as partial/manifest-incomplete. SignerProvider separates cursor and manifest purposes, authenticates the canonical service-derived binding with the envelope, and fails closed without an appropriate provider. Algorithm, custody, rotation, archival verification and storage controls remain pre-execution gates.
6. Integration fixtures use at least two tenants with identical operation/subject/provider identifiers to prove tenant scoping at every boundary.

Appendix B. Source basis and assumptions

Primary design basis: M02 Detailed Requirements — Event and Operation Foundation, Approved 1.0, 28 August 2026.

Supporting sources and assumptions:

1. M01 Technical Design — Tenant Context and Data Access, Draft 0.2, 26 August 2026, supplies the mandatory tenant context, transaction gateway, role, pool, and PHI-safe telemetry boundaries.
2. Track B Event Tracking Requirements v2.4 and architecture decisions ADR-003, ADR-005, and ADR-007 provide historical context; approved M02 requirements control refined semantics.
3. Cloud SQL PostgreSQL 17 is the authoritative event store. Shared reference/control data follows the M01 classification and access model.
4. External callbacks are at least once and may arrive out of order. Provider lookup, idempotency, echoed-reference, and timestamp capabilities remain adapter-specific and require M02-OI-005 evidence.
5. M03 owns validated durable intake; M05 owns staff work; M06 owns scheduling and mutable reconciliation; M11 owns correlation governance and observability operations.
6. Exact retention, legal-hold, privileged-change, service-objective, and capacity values remain gated open items and are not invented by this design.

Appendix C. Design review checklist

Review	Approval question	Required evidence / outcome
Architecture	Are D-02–D-09 accepted and reflected in ADR-011?	Approved ADR scope, decision owner, and supersession record.
Data architecture	Are table boundaries, constraints, indexes, contract rollout, and rebuild semantics implementable?	Schema review, migration plan, explain-plan targets, and compatibility matrix.
Security	Do roles, append-only enforcement, correction authority, dedupe conflict, replay, and integrity controls fail safe?	Threat-model sign-off, grants manifest, negative-test plan, and OI-010 decision.
Privacy / compliance	Are metadata, retention, legal hold, correction, export, and disposition minimum necessary and lawful?	Classification manifest, retention/disposition policy, and privileged-audit approval.
SRE	Are handoff recovery, monitoring, capacity, fairness, and rollback operable?	OI-009 objectives, dashboards/alerts, runbooks, load and recovery test plan.
Integration / product	Are provider idempotency, lookup, callback identity/time, conflict, and resend behaviors supported?	OI-005 capability evidence and approved safe fallback per adapter.
M03/M05/M06/M10/M11 owners	Do versioned handoff/query contracts preserve each module's ownership and converge under replay?	Consumer contract tests, deterministic keys, compatibility versions, and named on-call owners.

Approval result. Approved as the M02 implementation design baseline on 30 August 2026 after closure of T1-T8 and V1-V5. This approval does not waive ADR-011, schema-freeze evidence, provider capability, retention, OI-009 service objectives, recovery exercises, or pilot/production readiness gates.

Appendix D. v0.3 review disposition and approval

ID	Disposition	v0.3 resolution
T1	Accepted - high	operation_registry row lock serializes complete-set projection and conflict/outbox creation; barrier test required.
T2	Accepted - high	V1 verifies configured-provider fail-closed behavior and version lookup continuity where applicable. Cloud KMS, producer-registration and rotation-specific behavior in the following requirement is deferred to OI-009: Single database-controlled active write version with compatible lookup retention and control-row lock during rotation.
T3	Accepted with implementation refinement	Approved procedures deny raw runtime mutation; deferred constraint trigger independently enforces patient_id-or-link at COMMIT.
T4	Accepted	Nullable patient_id restored; patient and link indexes receive explain-plan targets. No requirements deviation remains.
T5	Accepted	V1 verifies configured-provider fail-closed behavior and version lookup continuity where applicable. Cloud KMS, producer-registration and rotation-specific behavior in the following requirement is deferred to OI-009: Cloud KMS-protected tenant/version key custody, bounded cache, fail-closed behavior, rotation, backup, and loss recovery specified.
T6	Accepted	listStaleIntents authoritative anti-join and M06 detector responsibilities specified.
T7	Partially accepted	Removed 'where safe'; provider capability declares namespace uniqueness scope and activates a precise partial unique index.

ID	Disposition	v0.3 resolution
T8	Partially accepted	Paged fair tenant sweep, metrics, OI-009 bounds, and architecture review trigger specified.
Minor 1	Clarified	Application code orchestrates one transaction; runtime executes purpose-specific procedures and has no raw table mutation grants.
Minor 2	Accepted	Retention-locked encrypted exports and signed PHI-free manifests now define the proposed compensating control.
V1	Closed - high	m02_lock_operation and m02_begin_key_scope are narrow SECURITY DEFINER gateways owned by a NOLOGIN role; runtime keeps EXECUTE only, direct locks/UPDATE remain denied, safe search_path and transaction-lifetime lock semantics are explicit.
V2	Closed - low	PHI-free live producer registrations plus deployment-generation attestation are a mandatory precondition enforced by m02_rotate_key_version.
V3	Closed - low	Null-operation observations are exempt from the registry lock, create no operation projection, and remain in authoritative events/links and the subject timeline with owning-domain atomic state.
V4	Closed - low	idx_patient_events_stale_intent_scan is named with exact predicate/order and receives a release-gating explain-plan test.
V5	Closed - note	Cloud KMS is identified as a hard cold-process dependency with OI-009 signals, warm/cold outage tests, retained-key recovery, and restore drills.
Approval	Approved	No open critical or high design-review findings. Approved implementation baseline; separately owned schema-freeze, pilot, and production-readiness gates remain.